



Java Multithreading

Section 1: Learn

Introduction to Multithreading

Multithreading in Java allows **concurrent execution** of two or more threads for maximum CPU utilization.

Each **thread** is a **separate path of execution**, and multithreading is used to perform multiple tasks simultaneously.

Benefits of Multithreading

- Better CPU utilization
 - Improved application performance
 - Enables background tasks (e.g., animations, downloads)
-

Thread Life Cycle

State	Description
New	Thread is created but not started
Runnable	Thread is ready to run
Running	Thread is executing
Blocked/ Waiting	Thread is paused for resource or signal
Terminated	Thread has completed or exited



Thread Class vs Runnable Interface

Feature	Thread Class	Runnable Interface
Inheritance	Extends Thread	Implements Runnable
Flexibility	Less (no multiple inheritance)	More flexible
Usage	<code>Thread t = new MyThread();</code>	<code>Thread t = new Thread(obj)</code>

Using Thread Class:

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running");  
    }  
}
```

Using Runnable Interface:

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable running");  
    }  
}
```

Synchronized Methods and Blocks

Used to prevent **thread interference** and **data inconsistency** when multiple threads access shared resources.



Synchronized Method:

```
public synchronized void display() {  
    // only one thread can execute at a time  
}
```

Synchronized Block:

```
synchronized(obj) {  
    // critical section  
}
```

Daemon Threads

- Background threads that run **in service of other threads**
- JVM exits when **only daemon threads** are left

```
Thread t = new Thread(() -> System.out.println("Daemon"));  
t.setDaemon(true);  
t.start();
```

Section 2: Practice

Create and Start a Thread

```
class HelloThread extends Thread {  
    public void run() {  
        System.out.println("Hello from thread");  
    }  
}  
  
public class Main {
```



```
public static void main(String[] args) {  
    HelloThread ht = new HelloThread();  
    ht.start();  
}  
}
```

Runnable with Lambda

```
Runnable r = () -> System.out.println("Running in Runnable");  
Thread t = new Thread(r);  
t.start();
```

Section 3: Know More (FAQs)

Can a thread be restarted?

No. Once a thread is terminated, it cannot be started again.

Which is better: Thread or Runnable?

Runnable is preferred when your class extends another class (supports multiple inheritance via interface).

What happens if two threads access the same variable?

If unsynchronized, it can lead to **data inconsistency** or **race conditions**.

Can a thread run without start()?

No. You must call **start()** to move the thread to **Runnable** state.

How to check if a thread is alive?

Use **t.isAlive()** method.